

TITLE PAGE

- **Problem Statement ID – Question No. 3 (QA)**
- **Problem Statement Title- Autonomous Production Choke Control System for Naturally Flowing Oil Wells**
- **Theme- Industrial Automation / Smart Process Control & Optimization**
- **PS Category- Software**
- **Student Name – Arpit Mittal**
- **Student ID - 23BCI0044**

IDEA TITLE

Autonomous Predictive Choke Controller (APCC) with Active Safety-Constraint Enforcement

- **Detailed explanation:** Software controller that continuously regulates the production choke (0-100%) to hit target oil flow rate without manual intervention; combines a Power-Law steady-state model with a First-Order-Lag dynamic filter to predict future flow (Q) and pressures (WHP, FLP, BHP)
- **How it addresses the problem:** Removes manual, trial-and-error choking; evaluates 21 candidate moves (within +/-5% ramp) against safety limits before acting; when a target is physically unreachable, it automatically settles at the Maximum Achievable Safe Flow Rate
- **Innovation and uniqueness:** Look-ahead safety shield checks predicted (not lagging) states to pre-empt pressure surges; model validated against the official simulation dataset with $R^2 = 0.9959$ for flow-rate dynamics

TECHNICAL APPROACH

- **Technologies used:** Python 3.11 (typed, object-oriented, modular); NumPy for numerical modelling; Pandas for data pipeline/logging; Matplotlib/Seaborn for diagnostic dashboards; abstract SimulatorInterface for plug-and-play swap between simulation and real-time SCADA/API endpoints
- **Methodology / control flow:**
 1. Measure & Predict - read sensor states (Q, WHP, FLP, BHP) into the verified power-law dynamic model (τ ~6.2 hr for Q; 9-11 hr for pressures)
 2. Generate Candidates - choke moves at 0.5% resolution, bounded to +/-5%/hr ramp limit
 3. Safety Filter - discard candidates that predict WHP (200-3000 psi), FLP (150-2500 psi) or BHP (2000-5000 psi) outside safe envelopes
 4. Optimal Selection - pick the feasible step minimizing $|Q_{\text{predicted}} - Q_{\text{target}}|$, else fall back to max safe Q

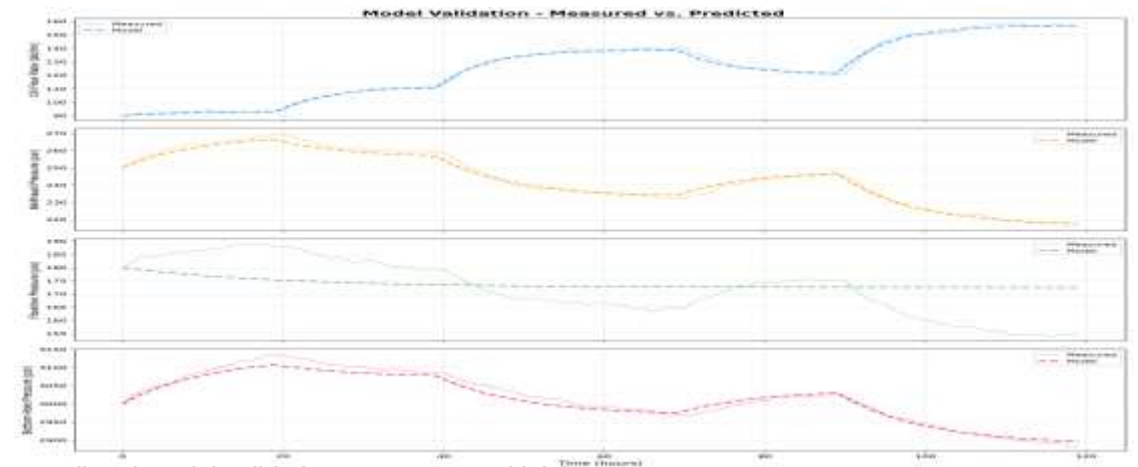
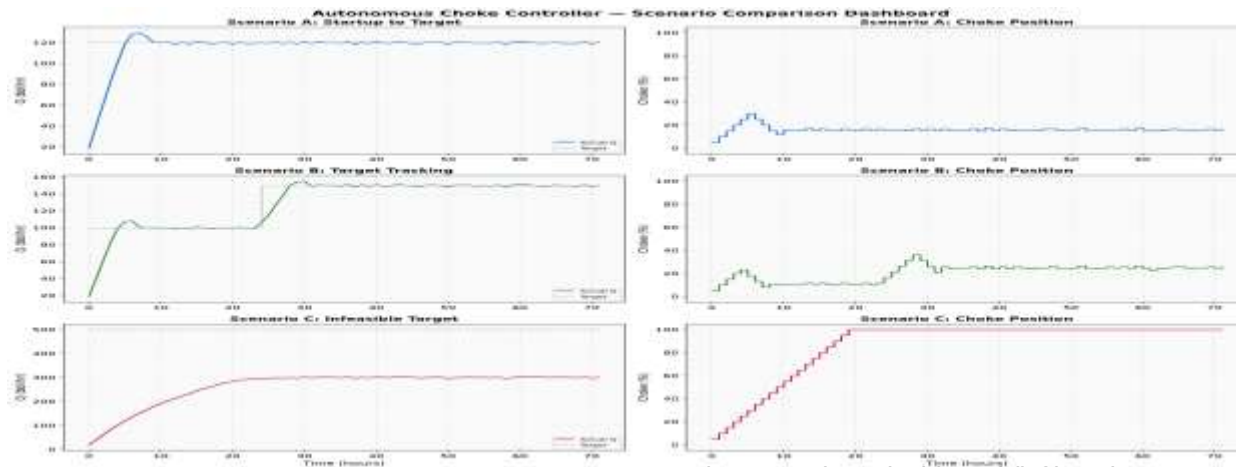
FEASIBILITY AND VIABILITY

- Feasibility: candidate generation evaluates ~21 points per step, computing the optimal control command in under 5 ms with no third-party solver; runs as pure software over existing PLC/DCS digital interfaces with zero hardware modification
- Challenges & risks: model-plant mismatch as reservoir dynamics drift with GOR shifts and pressure depletion; sensor noise/disturbance could cause "hunting" (unnecessary oscillation around the choke setting)
- Strategies: built-in $\pm 5\%$ /hr ramp-rate dampening prevents overshoot and stabilizes transients; periodic automated re-identification pipeline (`model_identification.py`) recalibrates tau and power-law coefficients directly from SCADA logs as well conditions evolve

ARTIFACTS

- Relevant artifacts:
 - Core control-loop code (right) - controller.py
 - Scenario dashboard - overlays Startup, Target Tracking & Safety Fallback (screenshots below)
 - Model validation vs. organizer dataset ($R^2 = 0.9959$)

```
# controller.py - core decision loop
for u_cand in self._generate_candidates(current_choke):
    pred = self.model.predict(current_state, u_cand)
    if self._is_feasible(pred):
        best_choke = self._select_optimal(pred, target_Q)
# tracks target, or falls back to max safe Q
```



Scenario comparison dashboard (left) and measured-vs-predicted model validation, $R^2 = 0.9959$ (right)

RESEARCH AND REFERENCES

- **Multiphase well mechanics:** Gilbert & Ros production choke-flow correlations (power-law and square-root formulations for sub-critical flow)
- **Process control theory:** Seborg et al., "Process Dynamics and Control" - First-Order-Plus-Dead-Time (FOPDT) modelling and discrete-sampling MPC principles
- **Reference data:** Validated against the official organizer-provided simulation dataset (Autonomous_Choke_Control_Simulated_Dataset.csv)
- **Codebase:** Modular Python architecture with abstract base interfaces, open-loop step-test suites, dynamic parameter regressors, and closed-loop validation engines